The problem being addressed

Platform core is addressing

Room/lobby lifecycle

- **Create**
  - Players click **Create Game** button
  - 4 letter code is generated
  - Maximum of 10 players in a lobby (adjustable)
  - User sets lobby to private or public
  - Game master (creator of game) can remove players from lobby

- **Join**
  - User clicks **Join Game** button
  - Players can join public lobbies without a code
  - To join private lobbies users must input 4 letter code

- **Leave**
  - User clicks **Leave Game** button
  - Returned to the main menu
  - Removed from lobby

Turn progression and game state management

Turn progression (WebSocket-Based)

- Implement WebSockets to support real-time turn progression
  - Use WebSockets so both players receive game updates instantly
  - Avoids constant polling (repeated API requests)
  - Ensures turn changes feel immediate and smooth

- Use FastAPI native WebSockets
  - Leverage FastAPI's built-in WebSocket support (`/ws/game/{game_id}`)
  - Keeps backend stack simple
  - Easy to integrate with existing FastAPI routes and game logic

Core turn flow (Server-Authoritative)

- Server controls turn progression
  - Client sends a move request
  - Backend validates the move and updates turn order
  - Prevents cheating and client-side desync issues

- WebSocket gameplay loop
    - Current game state is loaded (from MongoDB or API)
    - Active player submits move
    - FastAPI validates: correct player, valid move, game is still active
    - Backend updates game state
    - Backend switches turn to next player
    - Updated state is broadcast to all players in the room

Game room / session management

- Use game-specific WebSocket rooms
    - Each game has a unique `game_id`
    - Players connect to `/ws/game/{game_id}`
    - Only players in that game receive updates for that match
- Track connected players
    - Maintain a connection manager in FastAPI
    - Map `game_id -> connected sockets`
    - Handle disconnects cleanly (remove stale sockets)

MongoDB integration (Persistence)

- Store game state in MongoDB
    - Current board/state
    - Active player turn
    - Move history
    - Game status (`waiting`, `in_progress`, `finished`)
    - Player IDs / usernames
- Why MongoDB is important
    - Supports reconnects (game state can be restored)
    - Prevents data loss if server restarts
    - Allows game history / replay / analytics later
- Suggested pattern
    - WebSocket handles live gameplay events and turn updates
    - REST endpoint fetches saved state on refresh/reconnect
    - MongoDB stores the source of truth

Validation rules (Turn-based)

- Validate every move on the backend
    - Is it the player's turn?
    - Is the move legal according to game rules?
    - Is the game already finished?
    - Is the player actually part of this game?
- Reject invalid actions

- Return `move_rejected` with reason
- Do not update game state or switch turn

Reconnect / refresh handling

- Support page refresh without breaking the game
  - Frontend calls REST endpoint to fetch latest game state
  - Frontend reconnects to WebSocket room
  - Prevents losing match progress after refresh
- Optional improvement
  - Store player session/token so they rejoin as the correct player automatically

Error handling / reliability

- Notify other player if someone disconnects
- Optionally pause game or start timeout countdown
- Handle duplicate or late messages
  - Ignore moves if they arrive after turn already changed
  - Use turn number / version number to prevent state conflicts
- Fallback plan
  - If WebSocket drops, frontend attempts reconnect
  - Reload game state after reconnect to resync

Security / access control

- Authenticate WebSocket connections
  - Pass auth token when connecting
  - Verify token in FastAPI before accepting socket
- Restrict actions
  - Only players in the match can send `make_move`
  - Spectator mode (if supported) should be read-only

Optional future enhancements

- Turn timer (server enforces timeout / auto-skip)
- Spectator mode (read-only WebSocket updates)
- In-game chat (reuse same WebSocket connection with `chat_message`)

Core data models, state updates, error handling

Core data models

- **Game**
  - `game_id`, lobby code, visibility (public/private)
  - Status (`waiting`, `in_progress`, `finished`)
  - Active player / current turn

    - Player list and host/game master id
    - Settings (max players, rules, timers if enabled)
    - Current game state (board/cards/round data)

- **Player**
    - `player_id`, display name
    - Game/lobby association
    - Connection status (`connected`, `disconnected`)
    - Role (host/player/spectator if added later)
    - Player-specific data (score, inventory, hand, etc.)

- **Move / action**
    - `move_id`, `game_id`, `player_id`
    - Action type + payload details
    - Timestamp
    - Saved for history/replay/debugging

- **WebSocket message model**
    - Standard format: `event`, `payload`, `timestamp`
    - Keeps frontend/backed communication consistent

State updates
- Server-authoritative updates
    - Frontend sends actions only
    - Backend validates and applies state changes
    - Backend broadcasts updated state to all players
- Persistence
    - Save state to MongoDB after important updates
    - Supports reconnects and recovery after refresh

Error handling
- Validation errors
    - Not your turn
    - Invalid move by rules
    - Player not in game
    - Game already finished
- Connection errors
    - Remove stale sockets on disconnect
    - Allow reconnect and resync from saved state
- State conflict protection

- Ignore late/duplicate moves after turn changes
- Optional: turn number / version check

WebSocket event types (client → server / server → client)

Client -> server events

- `join_game` (player joins game room)
- `make_move` (player submits a move)
- `end_turn` (optional, if separate from move)
- `player_ready` (optional, match start / ready check)
- `reconnect` (restore player session)
- `leave_lobby` (player leaves lobby/game)
- `kick_player` (host-only)
- `chat_message` (optional)

Server -> client events

- `game_state` (full updated game state)
- `turn_changed` (indicates whose turn it is now)
- `move_accepted` (move was valid and processed)
- `move_rejected` (invalid move / not player's turn)
- `game_started` (game has begun)
- `game_over` (win/loss/draw result)
- `player_disconnected` (optional)
- `player_reconnected` (optional)
- `error` (general error message/code)

Event format consistency

- Use a standard structure: `event`, `payload`, `timestamp`
- Backend validates event type + payload shape
- Invalid/missing fields return `error` event

How the design integrates with the rest of the platform

Client/UI Sub-Team

- WebSocket events update the game room UI in real time (board, turn status,

player status, messages)
    - Supports lobby screen, game room screen, and reconnect flow without polling

Rules + Validation Sub-Team
    - `make_move` events go through server-side validation before state updates
    - Turn progression only advances after a move is confirmed legal

Quality + Testing Sub-Team
    - Clear event flow supports end-to-end testing (create/join/start/move/reject/reconnect)
    - Consistent event formats make automated and demo-script testing easier

Integration + Release Sub-Team
    - Uses the same stack (React + FastAPI + MongoDB) with no extra Firebase service
    - Easier to maintain one build/run process, one README, and one release flow

Platform core / lobby integration
    - Lobby lifecycle (create/join/leave) feeds directly into game start and turn order
    - Lobby room code / `game_id` is reused for WebSocket game sessions

Backend + database integration
    - REST handles create/join/leave/fetch state
    - WebSocket handles live gameplay events and turn updates
    - MongoDB stores persistent game state for reconnects and recovery

<u>Key assumptions and trade-offs (Platform core)</u>

- Assumption: lobby codes are short and simple (4 letters)
    - Trade-off: easier for users to join, but higher chance of duplicate codes if many lobbies exist at once

- Assumption: max lobby size is capped (default 10, adjustable)
    - Trade-off: easier to manage UI/state and gameplay flow, but limits large-room use cases

- Assumption: host/game master controls the lobby
    - Trade-off: simpler moderation (kick/remove/start), but creates dependency on host behavior/availability

- Assumption: private/public lobby model is enough for access control
    - Trade-off: simple join flow, but weaker security than full invite/auth-based access

- Assumption: lobby lifecycle is handled before gameplay starts
    - Trade-off: cleaner separation between lobby and game systems, but requires clear handoff to game state initialization

- Assumption: one lobby maps to one active game session
    - Trade-off: easier tracking and integration (room code / `game_id`), but less flexible for multi-match rooms or rematch variants

- Assumption: players can leave at any time
    - Trade-off: improves usability, but platform core must handle edge cases (host leaving, empty lobby, in-progress game)

- Assumption: platform core owns room/session membership, not game rules
    - Trade-off: clearer team boundaries, but requires strong integration contracts with gameplay/validation teams